

# DOM Moderno (Javascript)

## Contenidos

DOM: Document Object Model

Búsquedas en el DOM

HTML/CSS desde el DOM

APIs interesantes de DOM

Inspirado en el bootcamp de  
**Manz.dev** todos los créditos a el.

# Document Object Model

## ¿Qué es el DOM?

HTML visto desde Javascript

Árbol de elementos HTML

Podemos modificarlo a nuestro gusto

Permite cambios dinámicos: «cuando el usuario...»

Muy subestimado (por usuarios de frameworks)





# DOM moderno

- Buscar elementos
- Crear elementos desde el DOM
- CSS (Clases/styles) desde el DOM
- Contenido desde el DOM
- Insertar HTML desde el DOM



## buscar elementos

|                                                |                          |
|------------------------------------------------|--------------------------|
| <code>.querySelector( selector CSS )</code>    | Primer elemento (o null) |
| <code>.querySelectorAll( selector CSS )</code> | Array (NodeList) o []    |
| <code>.closest( selector CSS )</code>          | Primer ancestro (o null) |

⚠ Evita métodos como `.getElementById()` o `.getElementsByName()`. Son más antiguos, menos flexibles y no tienen la potencia de búsqueda de CSS.

## crear elementos o atributos

|                                                   |                          |
|---------------------------------------------------|--------------------------|
| <code>.createElement( etiqueta HTML )</code>      | Elemento creado          |
| <code>.setAttribute( atributo , valor )</code>    | Añade un atributo HTML   |
| <code>.getAttribute( atributo )</code>            | Obtiene el valor         |
| <code>.toggleAttribute( atributo , force )</code> | Alterna (añade/elimina)  |
| <code>.removeAttribute( atributo )</code>         | Elimina el atributo HTML |

## clases del elemento

|                                       |                            |
|---------------------------------------|----------------------------|
| <code>.classList .add( clase )</code> | Añade una clase (o varias) |
| <code>.remove( clase )</code>         | Elimina clase              |
| <code>.toggle( clase )</code>         | Alterna (añade/elimina)    |

⚠ Evita usar `.className` a favor de `.classList`. Con `.className` reemplazas toda la lista de clases del elemento HTML o no mantienes anteriores.

## estilos del elemento

|                                                  |                                  |
|--------------------------------------------------|----------------------------------|
| ⚠ <code>.style .backgroundColor = valor</code>   | Obtiene/añade un estilo en línea |
| <code>.style .setProperty( prop , valor )</code> | Añade un estilo en línea         |
| <code>.style .getPropertyValue( prop )</code>    | Obtiene un estilo en línea       |
| <code>.style .removeProperty( prop )</code>      | Elimina un estilo en línea       |

⚠ Intenta usar siempre los métodos `.setProperty()` en lugar de usar la propiedad directa. Es más organizado y soporta variables de CSS.



## contenido del elemento

|                                               |                                |
|-----------------------------------------------|--------------------------------|
| <code>.textContent = texto</code>             | Inserta texto literalmente     |
| ⚠ <code>.innerHTML = contenido HTML</code>    | Modifica HTML (cuidado)        |
| <code>.setHTMLUnsafe( contenido HTML )</code> | .innerHTML moderno (modificar) |
| <code>.getHTML()</code>                       | .innerHTML moderno (obtener)   |

⚠ El método `.setHTMLUnsafe()` reemplaza al antiguo `.innerHTML`. Evita usar `.innerText` a favor de `.textContent`. Ojo al añadir HTML, puede ser inseguro.

## insertar elementos en el dom

|                                       |                                           |
|---------------------------------------|-------------------------------------------|
| <code>.prepend( elemento )</code>     | Inserta dentro de un elemento (al inicio) |
| <code>.append( elemento )</code>      | Inserta dentro de un elemento (al final)  |
| <code>.before( elemento )</code>      | Inserta antes de un elemento HTML         |
| <code>.after( elemento )</code>       | Inserta después de un elemento HTML       |
| <code>.remove()</code>                | Elimina un elemento HTML                  |
| <code>.replaceWith( elemento )</code> | Reemplaza un elemento HTML por otro       |

⚠ Evita usar `.appendChild()` en favor de estos métodos. Son más semánticos, más directos y mucho más flexibles.

## insertar elementos adyacentes

|                                                            |                          |
|------------------------------------------------------------|--------------------------|
| <code>.insertAdjacentElement( position , elemento )</code> | Añade un elemento        |
| <code>.insertAdjacentHTML( position , html )</code>        | Añade contenido HTML     |
| <code>.insertAdjacentText( position , html )</code>        | Añade contenido de texto |

|                                                    |                 |
|----------------------------------------------------|-----------------|
| <code>"beforebegin"</code>                         | Antes           |
| <code>&lt;div class="parent"&gt;</code>            |                 |
| <code>"afterbegin"</code>                          | Dentro (inicio) |
| <code>&lt;div class="child"&gt;&lt;/div&gt;</code> |                 |
| <code>"beforeend"</code>                           | Dentro (final)  |
| <code>&lt;/div&gt;</code>                          |                 |
| <code>"afterend"</code>                            | Después         |

## Buscar en el DOM

-  Métodos modernos de selección: `.querySelector*()`
-  Métodos «legacy» tradicionales: `.getElement*()` (limitado por ID, colecciones vivas...)

```
document          // Representa al árbol completo: el documento HTML

const element = document.getElementById("presentation"); //  Legacy (limitado a IDs)
const element = document.querySelector("#presentation"); //  Equivalente (y más potente)

const elements = document.getElementsByTagName("div");    //  Devuelve HTMLCollection
const elements = document.querySelectorAll("div");        //  Equivalente (NodeList)
const elements = document.getElementsByClassName("pato"); //  Devuelve HTMLCollection
const elements = document.querySelectorAll(".pato");      //  Equivalente (NodeList)
```



## Buscar en el DOM

-  Métodos modernos de selección: `.querySelector*()`
-  Métodos «legacy» tradicionales: `.getElement*()` (limitado por ID, colecciones vivas...)

```
const element = document.querySelector(".container"); //  Busca/selecciona el PRIMER elemento
//  Si no encuentra, devuelve null

const elements = document.querySelectorAll("div"); //  Busca/selecciona TODOS los elementos
//  Si no encuentra, devuelve []

// Puedes hacer búsquedas acotadas (no sobre todo el documento)
element.querySelector("p");
```

## Buscar en el DOM

-  Métodos modernos de selección: `.querySelector*()`
-  Métodos «legacy» tradicionales: `.getElement*()` (limitado por ID, colecciones vivas...)

```
// Imágenes que no tienen atributo "alt" (obligatorio)
const errorImg = document.querySelectorAll("img:not([alt])");

// Enlaces que lleven a una imagen PNG
const linkToImage = document.querySelectorAll("a[href$='.png']");

// Enlaces que estén hechos sobre una imagen
const imageWithLink = document.querySelectorAll("a:has(img)");

// Las celdas de la segunda fila de cada tabla de la página
const cells = document.querySelectorAll("table tr:nth-child(2) td");
```

## Buscar en el DOM

-  Métodos modernos de selección: `.querySelector*()`
-  Métodos «legacy» tradicionales: `.getElement*()` (limitado por ID, colecciones vivas...)

```
// Buscamos un elemento HTML concreto
const container = document.querySelector(".container");

// Ahora queremos continuar buscando pero hacia los padres, no hacia los hijos

// Busca el elemento más cercano (hacia arriba)
const ancient = container.closest(".page");
```



# Modificar contenido



## Modificar texto de la web

-  Preferir propiedad `.textContent`
-  Evitar `.innerText` (más lento, oculta partes no renderizadas)

```
const paragraph = document.querySelector(".container .post p");

paragraph.textContent // Devuelve el contenido de texto actual
paragraph.textContent = "Hola"; // Modifica el contenido actual por "Hola"

// 🛡️ Esto es muy importante en temas de seguridad
paragraph.textContent = "<div>Hola</div>"; // Inserta código HTML literalmente
```

## Modificar HTML

- ✗ Propiedades «legacy» tradicionales: `.innerHTML` y `.outerHTML`
- ✓ Métodos modernos: `.setHTMLUnsafe()` y `.getHTML()`
- 🟠 Método futuro: `.setHTML()` (solo en Chrome aún)

```
const paragraph = document.querySelector(".container .post");  
  
// Sirve tanto para leer como para modificar  
paragraph.innerHTML // '<p>Esto es un párrafo de texto</p>'  
paragraph.innerHTML = "<p>Hola, soy <strong>Alonso</strong>.</p>";  
  
// Confusa en ciertas situaciones. No soporta ciertos detalles modernos.
```

## Modificar HTML

- ✗ Propiedades «legacy» tradicionales: `.innerHTML` y `.outerHTML`
- ✓ Métodos modernos: `.setHTMLUnsafe()` y `.getHTML()`
- 🟡 Método futuro: `.setHTML()` (solo en Chrome aún)

```
const paragraph = document.querySelector(".container .post");

// Métodos modernos (más intuitivos)
paragraph.getHTML();           // '<p>Esto es un párrafo de texto</p>'
paragraph.setHTMLUnsafe("<p>Hola, soy <strong>Alonso</strong></p>");

// Deja claro que puede insertar HTML inseguro (innerHTML también lo es)
```

## Modificar HTML

- ✗ Propiedades «legacy» tradicionales: `.innerHTML` y `.outerHTML`
- ✓ Métodos modernos: `.setHTMLUnsafe()` y `.getHTML()`
- 🟡 Método futuro: `.setHTML()` (solo en Chrome aún)

```
const options = {
  sanitizer: new Sanitizer({
    comments: true, // Elimina comentarios HTML
    removeElements: ["script", "iframe"] // Elimina scripts e iframes
  })
}

document.body.setHTMLUnsafe("<p onmouseenter='alert(1)'>Hola amigo mío.</p>"); // Inseguro
document.body.setHTML("<p onmouseenter='alert(1)'>Hola amigo mío.</p>", options); // Seguro
```



## Atributos HTML

- ✗ Mediante propiedades `.id`, `.style` → Ojo, una propiedad != un atributo
- ✓ Mediante métodos `.setAttribute()`, `getAttribute()` o `.style.setProperty()`
- ✓ Extras: `.toggleAttribute()` y `.removeAttribute()`

```
const element = document.querySelector(".element");

element.nodeName           // Esto es una propiedad (la etiqueta HTML en mayúsculas)
element.getAttribute("nodeName") // Esto SI es un atributo HTML
// Si tuvieramos el atributo nodeName, tendríamos valores diferentes

const presentation = document.querySelector("#presentation");
element.id             // "presentation"
element.getAttribute("id") // "presentation"
// Es la misma porque ciertos atributos HTML los REFLEJA AUTOMÁTICAMENTE
```

# Atributos HTML

- ❌ Mediante propiedades .id, .style → Ojo, una propiedad != un atributo
- ✅ Mediante métodos .setAttribute(), getAttribute() o .style.setProperty()
- ✅ Extras: .toggleAttribute() y .removeAttribute()

```
const element = document.querySelector(".element");

// ❌ Forma rápida
element.id = "name";
element.style.backgroundColor = "indigo"; // Cambia la propiedad CSS background-color (inline style)

// ✅ Forma metódica
element.setAttribute("id", "name");
element.setAttribute("style", "background-color: red");
element.style.setProperty("background-color", "red");
```

## Atributos HTML

- ✗ Mediante propiedades .id, .style → Ojo, una propiedad != un atributo
- ✔ Mediante métodos .setAttribute(), getAttribute() o .style.setProperty()
- ✔ Extras: .toggleAttribute() y .removeAttribute()

```
// Caso especial: Booleanos
const video = document.querySelector("video");

video.controls = true;
video.setAttribute("controls", "");      // Atributo HTML booleano: false = no existe atributo

video.removeAttribute("controls");
video.toggleAttribute("controls");        // Si existe, lo elimina. Si no existe, lo añade
```

## Las clases HTML

- ❌ La propiedad `.style` (inline styles) y `.className` (clase como texto)
- ✅ El objeto `.classList` con métodos de clases

```
element.className = "parent element";           // ❌ Debes añadirlas directamente

element.classList.length                        // 2
element.classList.item(2);                     // "element"
element.classList.contains("rounded");         // false
element.classList.add("rounded");              // Añade la clase
element.classList.remove("rounded");           // Elimina la clase
element.classList.toggle("rounded");           // Añade (si no existe) o elimina (si existe)
element.classList.replace("rounded", "hide");  // Cambia "rounded" por "hide"
```

# Crear elementos con el DOM



## Crear elementos

- El método `document.createElement()`
- Podemos crear un sistema para simplificar
- [Fragmentos](#) en casos especiales (no queremos contenedor)

```
const div = document.createElement("div");
div.classList.add("duck-element");
div.textContent = "Protip: Don't use Tailwind!";

div.isConnected // false (no está en el DOM, está en memoria)
document.body.append(div);
div.isConnected // true (si está en el DOM)
```



## Crear elementos

- El método `document.createElement()`
- Podemos crear un sistema para simplificar
- `Fragmentos` en casos especiales (no queremos contenedor)

```
const createTag = (className = "element", tag = "div", options = {}) => {
  const element = document.createElement(tag);
  element.classList.add(className);
  if (options.text) element.textContent = options.text;
  if (options.html) element.setHTMLUnsafe(options.html);
  return element;
}

createTag("container", "p", { text: "Hello!" }); // <p class="container">Hello!</p>
```

## APIs inserción

- ❌ API de Nodos: La más vieja, clásica y genérica
- ✅✅ API de Element: La intermedia: Corta y práctica
- ✅ API de inserción adyacente: La más flexible

```
const parent = document.querySelector(".parent");  
// <div class="parent">Mensaje: </div>  
  
const tag = createTag("welcome", "p", { text: "¡Bienvenido, usuario!" });  
// <p class="welcome">¡Bienvenido, usuario!</p>  
  
parent.appendChild(tag);  
//                                     ↓ Lo inserta aquí (poco flexible)  
// <div class="parent">Mensaje: </div>
```

## APIs inserción

- ✗ API de `Nodos`: La más vieja, clásica y genérica
- ✓✓ API de `Element`: La intermedia: Corta y práctica
- ✓ API de `inserción adyacente`: La más flexible

```
const parent = document.querySelector(".parent");
const tag = createTag("welcome", "p", { text: "¡Bienvenido, usuario!" });
```

```
parent.before(tag);      // (↓) <div class="parent">Mensaje:</div>
parent.prepend(tag);     // <div class="parent">(↓) Mensaje:</div>
parent.append(tag);      // <div class="parent">Mensaje: (↓) </div>
parent.after(tag);       // <div class="parent">Mensaje:</div> (↓)
```

```
tag.remove();           // Elimina el elemento del documento HTML
```

## APIs inserción

- ✗ API de Nodos: La más vieja, clásica y genérica
- ✓✓ API de Element: La intermedia: Corta y práctica
- ✓ API de inserción adyacente: La más flexible

```
const tag = createTag("welcome", "p", { text: "¡Bienvenido, usuario!" });

// API de inserción adyacente
parent.insertAdjacentElement("beforeend", tag);
parent.insertAdjacentHTML("beforeend", `<p>Puedes insertar el HTML directamente</p>`);
parent.insertAdjacentText("beforeend", `O insertar textos directamente sin etiquetas HTML`);
```

# Plantillas HTML

- Etiqueta `<template>` de HTML → `template`

```
<template>
  <div class="card">
    <h1></h1>
  </div>
</template>

<div class="page">
  <!-- Contenido -->
</div>
```

```
const cats = [
  "Lea", "Grisu", "Rocky",
  "Giacomino Guardiano Dell Iperspazio", "Pimpi"
];
const page = document.querySelector(".page");
const template = document.querySelector("template");

cats.forEach(cat => {
  const html = template.content.cloneNode(true);
  html.querySelector("h1").textContent = cat;
  page.append(html);
});
```

# APIs relacionadas



## Tipos de DOM

- DOM: El «DOM real», el documento general que solemos utilizar siempre
- Virtual DOM: Un DOM en memoria que utilizan librerías como React
- [Shadow DOM](#): Caso particular del DOM → Shadow DOM

```
const element = document.querySelector(".container");

// Creamos un Shadow DOM en el elemento (se oculta el DOM real)
element.attachShadow({ mode: "open" });

// Ahora podemos modificar el DOM real (que se cambia pero no se ve)
element.setHTMLUnsafe(`<p>Contenido del Light DOM (DOM real)</p>`);

// ...y modificar el Shadow DOM (el que se ve actualmente)
element.shadowRoot.setHTMLUnsafe(`<p>Contenido del Shadow DOM</p>`);
```



# API de Animaciones del DOM

- Animaciones CSS (pero desde Javascript) → [WebAnimations](#)

```
const element = document.querySelector(".element");

element.animate({
  opacity: [1, 0],
  height: ["25px", "0"]
}, 2000);

element.animate({ /* ... */ }, {
  duration: 2000,
  fill: "forwards"
});
```

```
/* Animaciones CSS */
@keyframes hide {
  0% { opacity: 1 }
  100% { opacity: 0 }
}

@keyframes reduce {
  0% { height: 25px; }
  100% { height: 0; }
}
```

# API de CSS Highlight

- Posibilidad de dar estilo a fragmentos sin usar DOM → [CSS Highlight API](#)

```
<pre><code class="language-css">
  @keyframes reduce {
    0% { height: 25px; }
    100% { height: 0; }
  }
</code></pre>
<style>
  .language-css .css-at-rule {
    background: indigo;
    color: white;
    padding: 2px 6px;
  }
</style>
```

```
<!-- En códigos grandes, DOM crece MUCHÍSIMO -->
<pre><code class="language-css">
  <span class="css-at-rule">@keyframes</span>
  <span class="css-id">reduce</span> {
    <span class="css-percent">0%</span> {
      <span class="css-property">height</span>:
      <span class="css-number">25</span>
      <span class="css-unit">px</span>; }
    <span class="css-percent">100%</span> {
      <span class="css-property">height</span>:
      <span class="css-number">0</span>; }
  }
</code></pre>
```

# API de CSS Highlight

- Posibilidad de dar estilo a fragmentos sin usar DOM → [CSS Highlight API](#)

```
<pre><code class="language-css">
  @keyframes reduce {
    0% { height: 25px; }
    100% { height: 0; }
  }
</code></pre>
<style>
  ::highlight(css-at-rule) {
    --color: indigo;
    background: color-mix(in srgb, var(--color), white 90%);
    color: var(--color);
    padding: 2px 6px;
  }
</style>
```

```
// Obtenemos el elemento <code>
const code = document.querySelector("pre code");

// Obtenemos el nodo de texto (a bajo nivel)
// "\n @keyframes..." <- Observa
const textNode = code.childNodes[0];

// Creamos el rango que nos interesa
const range = new Range();
range.setStart(textNode, 3);
range.setEnd(textNode, 13);

// Le damos nombre
CSS.highlights.set("css-at-rule", new Highlight(range));
```

## API de mutaciones del DOM

- Observar mutaciones con la API de MutationObserver

```
const container = document.querySelector(".container");

const addRandomMessage = () => {
  const time = 1000 + Math.floor(Math.random() * 3000);
  container.textContent = `¡Mensaje recibido: ${time}!`;
  setTimeout(() => addRandomMessage(), time);
}

const observer = new MutationObserver(() => console.log("Cambio detectado en el contenedor."));
observer.observe(container, { childList: true });

setTimeout(() => addRandomMessage(), 3000);
```

## Plantillas HTML (con Lit)

- Sistema de plantillas ultraligero, núcleo de [Lit](#) → `pnpm install lit-html`
- v3.3.2:  → 1.71 MB  → 7.2KB  → 3.24KB

```
import { html, render } from "https://unpkg.com/lit-html"; // o "lit-html" si usas pnpm

const app = document.querySelector("#app");

const template = (name) => html`<h1>Hola, soy <strong>${name}</strong></h1>`;

render(template("Alonso"), app); // renderiza el contenido en el DOM
render(template("Patito"), app); // re-render (no sobrescribe, solo detecta lo que cambia)
```

## Plantillas HTML (con Lit)

- Sistema de plantillas ultraligero, núcleo de [Lit](#) → `pnpm install lit-html`
- v3.3.2:  → 1.71 MB  → 7.2KB  → 3.24KB

```
import { html, render } from "https://unpkg.com/lit-html";

const app = document.querySelector("#app");

const addButton = (text, action) => html`<button @click=${action}>${text}</button>`;

render(addButton("Click me", () => alert("Clicked!!")), app);
```

## Plantillas HTML (con Lit)

- Sistema de plantillas ultraligero, núcleo de [Lit](#) → `pnpm install lit-html`
- v3.3.2:  → 1.71 MB  → 7.2KB  → 3.24KB

```
import { html, render } from "https://unpkg.com/lit-html";

const app = document.querySelector("#app");

// Estado
let isOpen = true;

// Modificación + render/re-render
const toggle = () => {
  isOpen = !isOpen;
  renderApp(); // Re-render
};

// Renderizado de la app
// Ojo, es una función
const renderApp = () => render(html`
  <button @click=${toggle}>Toggle</button>
  <p>Status: ${isOpen ? "Open" : "Closed"}</p>
`, app);

renderApp(); // Render inicial
```



## Referencias

- [CheatSheet Javascript](#)
- [bootcamp.manz.dev](#)